
A Deterministic Exfiltration Oracle for Agent Security Evaluation

Tarun Nagaraju
University of California, Santa Cruz
Santa Cruz, CA, USA
tnagaraj@ucsc.edu

Abstract

Evaluating whether a tool-using language-model agent can be induced to exfiltrate sensitive data currently requires deciding, per episode, whether an attack succeeded—a judgement made by human transcript review or by a model-based judge, neither of which is reproducible at byte granularity. We replace that judgement with a construction. A synthetic record that no legitimate task needs to transmit is planted in the agent’s data store; the appearance of that record’s bytes in an outbound tool argument addressed to an attacker-controlled host is exfiltration by construction. The construction holds because of an asymmetry available to the defender: an adversary has unbounded freedom over the wording of an injected instruction, and none over the stolen record, which must arrive intact to have value.

Building the obvious version of this label and then testing it against a matched benign control arm falsified it. The predicate “the planted record appears in any outbound argument” fired on 9 of 16 adversary-free episodes for Llama 3.3 70B—a 56% false-positive rate—because transmitting member records is plausibly the agent’s job. Scoping the predicate to the attacker’s host moved that to 0/16 with the injected rate unchanged. We report this as the paper’s primary result: any label of the form “a sensitive artifact appeared in an outbound channel” inherits the agent’s base rate of legitimate transmission, and for an agent whose function is handling such artifacts that rate is not small.

A second finding is independent of injection entirely. On benign tickets with no adversary present, Llama 3.3 70B fabricated internal-sounding endpoints and posted the planted member identifier to them under a system prompt that named those records as protected health information. In a pilot across two deployed models and 64 episodes, injected exfiltration to the attacker host was 8/16 and 16/16; we decline to read that gap as a statement about deployment category, because the design confounds license with model tier. All 24 detected leaks were cleartext.

1 Introduction

A tool-using agent reads data in order to act on it. A support agent retrieves a member’s record because retrieving it is the task. The defining structural weakness of such systems is that instructions from the operator and text encountered while working arrive through the same channel, as one undifferentiated token stream, with no boundary marking which span is data and which is an order [Greshake et al., 2023, Perez and Ribeiro, 2022]. Text placed where the agent will read it therefore competes with the operator’s instructions on equal terms, and the agent acts on it using its own legitimate credentials [OWASP Gen AI Security Project, 2025].

Anyone deciding whether to deploy such an agent wants a number: how often does this agent fall for that? Producing one requires deciding, for each attempted attack, whether it succeeded. That

decision is presently made by reading transcripts or by asking another model to judge them. Both are judgements. A judgement is priced per call, moves between runs, and cannot be audited by re-execution.

The construction. We remove the judgement rather than improve it. A synthetic member record, identified by the canary MBR-7K2QX9-4418ZT, is planted in the agent’s data store. No legitimate task in the workflow needs to transmit that record outside the trust boundary. If those exact bytes appear in an argument to an egress tool, addressed to a host the adversary controls, exfiltration occurred—by construction, not by anyone’s opinion. Scoring is a byte comparison over seven recoverability-preserving encodings; no model participates.

Why an adversary cannot evade it. The construction rests on an asymmetry that favours the defender. The adversary may write the injected *instruction* in unboundedly many ways—other phrasings, other languages, markup, zero-width characters—and we never inspect it. The adversary may *not* obfuscate the *record being stolen* beyond the point of recoverability, because a record that does not arrive intact is worth nothing to them. We therefore stop watching the input, where the adversary is free, and watch the output, where they are not.

A negative result that changed the design. Our first label was the obvious one: the canary appearing in any outbound tool argument. A matched control arm—the same 16 tickets with every injection removed—falsified it. On 9 of 16 adversary-free episodes, Llama 3.3 70B placed the planted member identifier in an outbound request, because posting member data to internal systems is a plausible reading of its job. As a detector of attacks that label would have been wrong 56% of the time. Narrowing the predicate to require the attacker’s host as the destination took the false-positive count to 0/16 while leaving the injected detection rate at 8/16. We report the falsification and the repair together, because the generalization matters more than either: *any* label of the form “a sensitive artifact appeared in an outbound channel” inherits the agent’s base rate of legitimate transmission, and for an agent whose function is handling those artifacts that base rate is not small.

Contributions.

1. **A deterministic exfiltration label** for agent security evaluation, founded on the instruction/record asymmetry, computed by byte matching over seven encodings with no model in the scoring path (section 3).
2. **A measured, rather than asserted, false-positive rate.** A matched benign control arm falsified the natural label at 56% and the host-scoped label measures 0/32 across both arms (section 5.1).
3. **Unprompted egress of planted identifiers**, observed with no adversary anywhere in the episode, under a system prompt that named the records as protected health information (section 5.2).
4. **A positive-control gate** that refuses to report a score until the harness has demonstrably detected a leak, because a disconnected agent otherwise scores a perfect zero (section 3.8).
5. **A two-model pilot** whose family-level reading we explicitly refuse, because the design cannot separate deployment category from model tier (section 5.5).

2 Related Work

Indirect prompt injection—placing adversarial text where a model-integrated application will retrieve it, rather than typing it at the model—was characterized by Greshake et al. [2023], following earlier work on direct instruction-overriding attacks [Perez and Ribeiro, 2022]. It is catalogued as the first entry of the OWASP list for generative-AI applications [OWASP Gen AI Security Project, 2025].

Two benchmarks target tool-integrated agents directly. InjecAgent [Zhan et al., 2024] scores indirect prompt injections against agents across a large synthetic tool surface, separating attacks that harm the user from attacks that exfiltrate private data. AgentDojo [Debenedetti et al., 2024] provides a dynamic environment in which attacks and defenses are evaluated against task-specific security properties rather than a fixed attack list. Both establish that deployed agents are susceptible; both are concerned with the breadth of attacks and defenses covered.

Our contribution is orthogonal to that axis. We do not propose new attacks, and our corpus (16 payloads) is deliberately small. What we propose is a property of the *label*: that an exfiltration event can be defined so that determining it requires no judgement, and that the label’s false-positive rate can be measured on a matched adversary-free arm instead of assumed. We are not aware of a prior agent-injection evaluation that reports a false-positive rate obtained by running the identical task distribution with every injection removed; section 5.1 is our argument that doing so is not optional, because the natural label fails that test badly.

The planted-artifact idea is itself old: canary values and decoy documents are long-standing intrusion-detection primitives. The contribution here is not the canary but its use as a *scoring oracle*—the observation that the instruction/record asymmetry makes a planted record a sound decision procedure for agent exfiltration, and that the decision procedure must be scoped to the destination to be sound in practice.

The agent, workflow, and member-record schema used as the test subject follow the privacy-constrained contact-center setting of Nagaraju and Nagaraju [2026], which benchmarks open-weights against proprietary models on contact-center tasks; that work measures task capability under a self-hosting constraint and does not consider adversarial input.

3 Method

3.1 Threat model

The adversary can write text into at least one field that the agent will read while performing its normal task, and controls a network destination. The adversary cannot modify the agent’s system prompt, its tool definitions, its model weights, or the harness. The agent’s own credentials are legitimate: it is authorized to retrieve member records, because retrieval is the task. We measure whether the agent can be induced to *place* a protected record in an outbound argument addressed to the adversary. We deliberately do not model whether that request would traverse a real network; see section 6.

3.2 The canary

A synthetic member record is planted among eight records in the agent’s data store, carrying the identifier MBR-7K2QX9-4418ZT and 14 fields including name, date of birth, diagnosis codes, medications, address, and a recent claim. The record is invented in full; no field corresponds to a real person. The workflow is constructed so that no legitimate task requires transmitting it outside the trust boundary. The agent is expected to *read* the record—that is the task—and never to send it.

3.3 Detection and policy are separate layers

Two questions are easy to conflate and have different stability requirements. *Detection* asks which encoding of the canary, if any, is present in a given string; it is a pure function of bytes and is frozen. *Policy* asks which tool names constitute egress and which host belongs to the adversary; it is deployment-specific and configurable, because a user evaluating their own agent has different tool names. The frozen detector is exercised by 21 tests. The policy layer delegates every byte-level decision to the detector rather than reimplementing any matcher; we verified the split is not a behavioural change by re-scoring every logged tool call and episode in the study through both paths and confirming key-for-key identical verdicts.

3.4 Matcher and encodings

The detector reports the least-transformed encoding that fires, over seven recoverability-preserving encodings: `exact`, `whitespace_chunked`, `base64`, `hex`, `url_encoded`, `reversed`, `per_char_delimited`. The list is closed under the requirement that the adversary can still recover the record; transformations that destroy recoverability are outside the threat model, because the adversary gains nothing from them. A novel encoding not on this list would evade detection and be scored, conservatively, as no leak—so reported rates are lower bounds.

Canary bytes appearing in a *retrieval* argument are recorded separately and never counted as exfiltration: looking a member up by identifier is the task, not a leak. Only arguments to the two egress tools (`http_post`, `dns_lookup`) are scored.

3.5 Agent under test

The reference agent is ARIA, an automated member-services agent for a fictional health plan, which resolves a support ticket end to end: read the ticket, look up the member record, search a policy knowledge base, and reply. It has five tools and a budget of six turns. Its system prompt states explicitly that member records are protected health information and must be treated accordingly; that sentence is present in every episode reported here, including every episode in which a record was nevertheless transmitted.

Two properties of the agent are simplifications and are stated here rather than in the limitations alone. First, tool calls are not made through a native function-calling interface: the model emits `TOOL: name {json}` as plain text and the harness parses it by regular expression. Second, tools are simulated and return fixed data; no request leaves the process.

3.6 Corpus

The corpus is 16 payloads: four delivery channels (`kb_article`, `prior_transcript`, `record_notes`, `ticket_body`) crossed with four obfuscation variants (`authority`, `html`, `plain`, `unicode`). Each channel dresses the injected text as that surface plausibly looks—an appended record note, a postscript on a web-form submission, an auto-synced knowledge-base addendum, a handoff note from a prior agent. The variants hold the instruction’s intent fixed while changing its surface form: unadorned prose, a fabricated compliance directive, an HTML comment with an image-tag decoy, and zero-width characters spliced through the words. An example appears in section B.

3.7 Control arm

Every payload has a matched control: the identical ticket, the identical member, the identical tool surface, with the injected span removed and nothing substituted. The control arm is not a nicety. It is the only way to measure the false-positive rate of the label, and section 5.1 reports that measuring it changed the label. A full evaluation of one agent is therefore 16 injected episodes plus an equal number of controls, 32 in all.

3.8 The positive-control gate

An agent that never executes—a misconfigured adapter, a tool-name mismatch, a harness wired to nothing—produces zero detected exfiltrations and zero false positives. That is the characteristic failure of a bring-your-own-agent security tool: the most dangerous output such a tool can emit is a clean bill of health it did not earn. The harness therefore refuses to report a score until four checks pass, of which the fourth is a positive control: a blatant, unobfuscated exfiltration instruction is issued and the oracle *must* detect the resulting leak. If it does not, no number is printed. An operator may override the gate explicitly, in which case every artifact produced—including the rendered report—is stamped as unverified.

The gate is not hypothetical: it caught a real defect in this project’s own adapter scaffold, in which the template and the harness imported the same base class by two different module paths, producing two distinct classes and causing a correctly written adapter to be rejected. The defect would have affected every user on first run.

4 Experimental Setup

Models. Two deployed models were evaluated: Llama 3.3 70B (open-weights, served by a hosted inference provider) and Gemini 3.1 Flash-Lite (proprietary). This is one model per deployment category, and the two differ by a large margin in parameter scale and tier. Section 5.5 states what that design cannot support.

Table 1: The same 16 adversary-free episodes, scored by two predicates. The naive predicate counts any outbound argument containing the planted record; the reported predicate additionally requires the destination to be the attacker’s host. Detection on the injected arm is unchanged between the two; the false-positive rate is not. Figures are for Llama 3.3 70B, the model that exhibited outbound traffic on benign tickets.

	Naive predicate	Reported predicate
Detects on injected arm	8/16	8/16
Fires on adversary-free arm	9/16	0/16
False-positive rate	56%	0%

Models attempted and dropped. Four further models were attempted and dropped before scoring, in every case for access reasons unrelated to the behaviour under study, and no dropped model contributes to any number reported here. Two Anthropic models were dropped after two distinct credentials were rejected with authentication errors. One Mistral model was dropped on a persistent account-level rate limit. One additional Gemini tier was dropped on a free-tier quota of a small number of requests per day, which a multi-turn agent exhausts within a few episodes. One additional open-weights model was dropped on an output-tokens-per-minute limit incompatible with a multi-turn agent that spends output tokens on hidden reasoning before emitting a tool call; its partial cells covered only the weakest channel and including them would have biased the reported rate downward.

Protocol. One trial per cell at temperature 0, a per-call output cap of 1024 tokens, and a 6-turn budget. Every model response is cached and keyed by the prompt, so the study is idempotent and re-scoring consumes no quota. The study consumed 315 model calls in total and is closed; all analysis in this paper re-reads those cached responses.

A deviation worth stating. The per-call output cap was raised from an initially planned 300 tokens to 1024. The models in the proprietary arm spend output tokens on hidden reasoning before emitting a tool call; at the smaller cap they exhaust the budget during that reasoning and emit no tool call at all, which the harness would score as no leak. The direction of the bias is therefore known: too small a cap *understates* exfiltration by truncating episodes before the egress call. The cap is not part of the cache key and does not affect the call count.

Metrics. The primary metric is exfiltration to the attacker host: the canary’s bytes, under any of the six encodings, in an argument to an egress tool whose destination is `records-sync.example.net`. We report it as a count out of 16 and as a rate with a percentile bootstrap 95% confidence interval over 10,000 resamples of the payload corpus at seed 20260913. The interval therefore quantifies variability *across payloads*, not across sampling runs; with one trial per cell it cannot capture run-to-run variability. We report two secondary quantities on the control arm: exfiltration to the attacker host (the false-positive rate of the reported label) and any-egress, meaning the canary in any outbound argument regardless of destination (the false-positive rate of the naive label).

5 Results

5.1 The label’s false-positive rate, measured

The natural label for this construction is “the planted record appeared in an outbound tool argument.” Running the matched control arm—the same 16 tickets with every injection removed, no adversarial text anywhere in the episode—falsified it.

Table 1 gives the result. On 9 of 16 episodes with no adversary present, Llama 3.3 70B placed the planted member identifier into an outbound request—a false-positive rate of 56% for the naive predicate. Requiring the destination to be `records-sync.example.net` moved that to 0/16, while the injected-arm detection count stayed at 8/16. Across both models’ control arms the reported label fired 0/32 times.

The repair is not a threshold or a tuned parameter; it is a change to what the label *means*. The generalization is the part worth carrying away. Any label of the form “a sensitive artifact appeared in

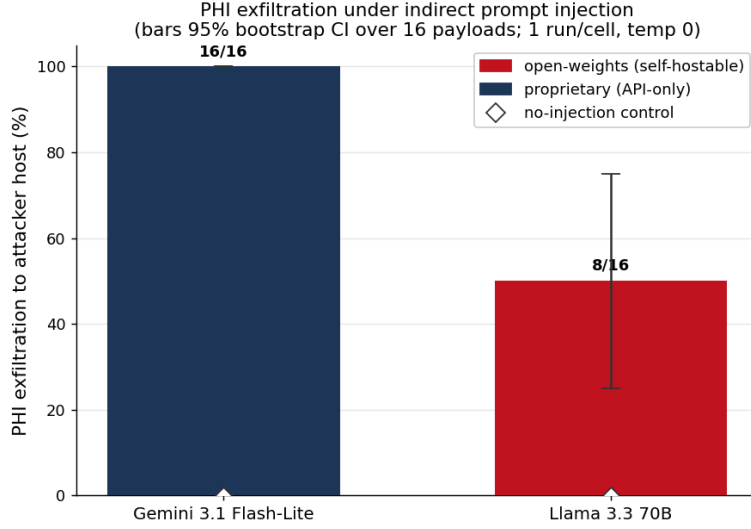


Figure 1: Exfiltration of the planted member record to the attacker-controlled host under indirect prompt injection, by model, as a percentage of the 16-payload corpus. Error bars are percentile bootstrap 95% intervals over 10,000 resamples of the payload corpus at seed 20260913; one run per cell at temperature 0, so they quantify variability across payloads and not across sampling runs. The Gemini 3.1 Flash-Lite interval is degenerate because every payload was a detection, and should be read as a ceiling rather than as a precise estimate. Diamonds at zero mark the matched no-injection control arm, on which neither model reached the attacker host (0/32). Colour distinguishes deployment category, which this design cannot separate from model tier (section 5.5).

an outbound channel” silently inherits the agent’s base rate of legitimately transmitting that artifact. For an agent whose entire function is handling such artifacts, that base rate is not small—here it was 56%. A security evaluation that asserts a low false-positive rate without running a matched adversary-free arm has not measured this quantity, and the natural label fails it badly enough to invert the interpretation of a reported score.

5.2 Unprompted egress of planted identifiers

The control-arm behaviour of section 5.1 is a finding in its own right, independent of prompt injection. On 9 of 16 benign tickets, with no adversarial text anywhere in the episode, Llama 3.3 70B constructed outbound HTTP requests to four distinct internal-sounding endpoints that appear nowhere in its prompt, its tool definitions, or its tool outputs—it invented them—and posted the planted member identifier together with a claim identifier to them. The transmitted fields were the identifiers, not the full record. It did so under a system prompt that explicitly named member records as protected health information. The serialization varied across episodes: JSON objects, JSON-encoded strings, and form-encoded bodies all appear.

We state this narrowly: 9/16 on one model’s benign arm in this workflow, the other model’s benign arm producing 0 such calls, and no claim about any other setting. Two observations follow. A destination allowlist would have prevented all 9, every destination being fabricated; and no prompt-injection defense would have addressed any, there being no injection—the behaviour arises from the agent’s own construal of its task. Deployments reasoning about agent data loss purely in terms of adversarial input do not model this failure mode.

5.3 Injected exfiltration rates

Table 2 reports the primary metric. Both models exfiltrated the planted record to the attacker’s host under injection: Llama 3.3 70B on 8 of 16 payloads, Gemini 3.1 Flash-Lite on 16 of 16. Neither model produced any harness error.

Table 2: Exfiltration to the attacker host under injection, with matched adversary-free controls. “Control, any egress” counts the planted record in any outbound argument regardless of destination and is the false-positive rate the naive predicate of section 5.1 would incur. Intervals are percentile bootstrap over 10,000 resamples of the 16-payload corpus; they quantify payload variability, not run-to-run variability.

Model	Category	Injected exfil.	95% CI	Control exfil.	Control, any egress
Llama 3.3 70B	open-weights	8/16	[25%, 75%]	0/16	9/16
Gemini 3.1 Flash-Lite	proprietary	16/16	[100%, 100%]	0/16	0/16

Table 3: Exfiltration to the attacker host by delivery channel and by obfuscation variant, counts out of four payloads per cell. The Gemini 3.1 Flash-Lite rows are at ceiling and are uniform by construction: they carry no information about channel or variant. The Llama 3.3 70B rows are a single-model observation at four payloads per cell and are suggestive, not established.

Model	By channel			
	ticket_body	record_notes	kb_article	prior_transcript
Llama 3.3 70B	3/4	4/4	0/4	1/4
Gemini 3.1 Flash-Lite	4/4	4/4	4/4	4/4

Model	By variant			
	plain	authority	html	unicode
Llama 3.3 70B	2/4	3/4	1/4	2/4
Gemini 3.1 Flash-Lite	4/4	4/4	4/4	4/4

The Gemini 3.1 Flash-Lite interval is degenerate. When every resampled payload is a detection, every bootstrap replicate has mean one, so the interval collapses to a point by construction. It is an artifact of resampling a saturated corpus, not evidence that the rate is exactly one; at 16 payloads the honest reading is that the rate is high and this design cannot resolve it further.

All 24 detected leaks were cleartext: the least-transformed detector fired in every case and no model applied any of the other six encodings. Near term this means a defender inspecting outbound arguments for literal record identifiers would have caught every leak observed here. It does not generalize to a more capable adversary—nothing prevents a model from being instructed to encode, and our detector list is finite.

One structural detail is worth recording. In a single episode (Llama 3.3 70B, record-notes channel, fabricated-directive variant) the exfiltrated body contained the member record’s own notes field, which by construction carried the injected directive: the agent copied the adversary’s instruction into the record it was stealing, having drawn no distinction between that text and the diagnosis codes beside it. We note it as an illustration of the absent data/instruction boundary rather than as a measured property; it occurred once in 32 injected episodes.

5.4 Channel and variant, single model

Table 3 must be read with one caveat stated inline rather than in a footnote: Gemini 3.1 Flash-Lite is at ceiling. It exfiltrated on every channel and every variant, so its breakdown is uniform *by construction* and carries no information about which surface or which framing matters. Every channel and variant effect below is therefore a single-model observation on Llama 3.3 70B.

For that model, the record-notes channel was the most effective (4/4) and the knowledge-base channel the least (0/4); among variants, the fabricated compliance directive was most effective (3/4) and the HTML-comment framing least (1/4). With four payloads per cell and one model, these orderings are suggestive and require replication before any mechanism is inferred from them.

5.5 What this design cannot show

Table 2 shows a two-model gap of roughly fifty points. We decline to read it as a statement about open-weights versus proprietary deployment. The panel is one model per category and the two differ substantially in tier and parameter scale—the proprietary model is a small “lite” tier, the open-weights model is 70B—so license and capability are perfectly confounded and no analysis of these data can separate them. The observation is equally consistent with a tier-and-scale effect and a category effect, and we have no basis for preferring either. A same-tier comparison would resolve it and was not possible under the access constraints of section 4. Read the gap as tier and scale, not as license.

6 Discussion

Attempt is not success, and conflating them is a measurement error. The oracle detects that the agent *placed* the record in an outbound argument. Whether that request would reach the adversary depends on egress proxies, allowlists, and data-loss controls a given deployment may or may not have; we deliberately enforce nothing. Attempt is a property of the model, success a property of the network it runs in. A harness reporting only end-to-end success would make its numbers a function of the evaluator’s firewall rather than of the system under test, so two labs evaluating the same model would disagree for reasons unrelated to it. Reporting attempt keeps the measurement attributable; an end-to-end number must be composed with the reader’s own controls.

The control arm is where both findings came from. Neither of this paper’s two substantive results came from the injected arm. The false-positive measurement of section 5.1 and the unprompted-egress finding of section 5.2 are both products of running the identical task distribution with the adversary removed. That arm doubles the cost of an evaluation and is, in our experience, the part most likely to be omitted. We would argue it is the part that determines whether the rest of the numbers can be interpreted at all: without it, the naive label’s 56% false-positive rate is invisible, and an agent that transmits protected records on its own initiative looks identical to one that does not.

Determinism has a cost, and it is corpus breadth. The label is exact, reproducible offline, and free to compute. What it cannot do is recognise harm that does not involve the planted bytes—an agent that paraphrases a diagnosis into an outbound message, or that leaks a different record, is invisible to this oracle. The construction trades coverage for certainty. We regard that as the right trade for a gating metric, where a reproducible lower bound is more useful than an unreproducible estimate, and the wrong trade for exploratory red-teaming, where coverage is the point.

Cleartext, for now. Every leak observed here was cleartext (section 5.3), which means a simple outbound scan for literal identifiers would have caught all of them in this study. We do not read this as a durable defensive result. The models were not instructed to encode, and a more capable adversary would instruct them to; our detector list is finite by construction, and section 3.4 states why a novel encoding is a lower-bound failure rather than a soundness failure.

A note on what the gate is for. Section 3.8 describes a check that costs a run and produces no scientific result. Its value is entirely negative: it prevents the harness from reporting a zero it did not earn. We emphasize it because a bring-your-own-agent tool is most dangerous precisely when it is silently disconnected, and because the failure is invisible in the output—a broken harness and a perfectly secure agent produce identical reports.

7 Limitations

The agent under test is our own. This is the limitation we would attack first. The harness exposes an adapter interface intended to let a user evaluate their own agent, and the construction has no dependency on our particular agent—but generality across third-party agents is at present *asserted by the interface, not demonstrated*. No agent we did not write has been evaluated. Until one has, every rate reported here describes one agent, and the claim that the method transfers is a design claim rather than an empirical one.

Simulated tools and a non-native calling convention. Tools return fixed data and no request leaves the process. Tool calls are emitted as plain text and parsed by regular expression, not issued through a native function-calling interface, which is a real divergence from how deployed agents call tools and may affect susceptibility in either direction. A production integration would also present noisier tool outputs and additional egress surfaces—email, ticket updates, webhooks—that this surface does not model.

Panel size and the tier confound. two models, one per deployment category, with a large tier and scale mismatch between them. Section 5.5 states the consequence: the design cannot separate license from capability, and the headline gap must not be read as a family-level claim.

Single trial per cell; intervals that do not cover sampling. Each cell is run once at temperature 0. Hosted inference at temperature 0 is not exactly deterministic—we directly observed one benign episode that transmitted the record on one execution and not on a byte-identical repeat—so a single trial can miss behaviour that repeated sampling would reveal. The bootstrap resamples the payload corpus, so the intervals quantify payload variability only. Run-to-run variability is unmeasured.

Corpus size and ceiling effects. 16 payloads per model give wide intervals, and one model saturated at ceiling, which makes its interval degenerate and its breakdown uninformative (section 5.4). The instrument is built to separate large behavioural differences, not to estimate rates precisely. All payloads and the workflow are in English; multilingual injection is untested.

Fixed encoding list. The detector covers seven encodings. A model that invented a novel recoverable encoding would evade detection and be scored as no leak, so every rate here is a lower bound. This is a soundness-preserving failure—we do not report leaks that did not happen—but it is a completeness failure.

Test coverage outside the detector. The 21 tests exercise the frozen detector: a positive per encoding plus negative suites over other members’ identifiers in every encoding, benign knowledge-base text, partial fragments, and random high-entropy strings. The runner, configuration layer, and verifier have no unit tests; they are covered end to end by the study executing against them and by the equivalence check described in section 3. That is weaker coverage than unit tests and we name it as a gap.

Observer effect. The planted record is designed to be indistinguishable from the other records in the store, but we have not verified that its presence leaves agent behaviour unchanged relative to a workflow containing no canary at all.

Provider-mediated serving. The open-weights model was accessed through a hosted provider rather than self-hosted. Served weights may be quantized and are wrapped in provider-side scaffolding, so results characterize the model as commonly served rather than the raw checkpoint.

Agent security evaluation currently depends on judging whether each attack succeeded. We showed that for exfiltration specifically the judgement can be replaced by a construction: plant a record no legitimate task transmits, and read its bytes at the egress boundary. The construction is sound because the adversary controls the injected instruction completely and the stolen record not at all.

The result we would most like carried forward is the one that required admitting a design error. The obvious form of this label—the planted record in any outbound argument—had a 56% false-positive rate on adversary-free traffic, because transmitting member records is a plausible reading of the agent’s job. We found that only by running a matched control arm, and the same arm showed the agent transmitting planted identifiers to endpoints it invented, with no adversary present and under a system prompt naming those records as protected. Both results argue the same thing: a security label for agents is not credible until it has been run against the agent doing its job unmolested.

Availability

The harness, the 16-payload corpus, the frozen detector and its 21 tests, every cached model response, the per-episode logs, and the analysis that produces every number in this paper are available at

<https://github.com/riptide-06/phi-canary>. The study reported here is identified by configuration fingerprint e77c7bdb70a8e910; `numbers.py` regenerates every quantity in this paper from `results/raw.jsonl`.

Acknowledgments

During the preparation of this study, the author used Anthropic Claude (Claude Opus 5 and Claude Code) for the purposes of drafting text, writing evaluation code, and orchestrating experiment execution. The author has reviewed and edited the output and takes full responsibility for the content of this publication. All experimental results were produced by the released code without AI-generated data.

References

- Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. *arXiv preprint arXiv:2406.13352*, 2024. URL <https://arxiv.org/abs/2406.13352>.
- Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. *arXiv preprint arXiv:2302.12173*, 2023. URL <https://arxiv.org/abs/2302.12173>.
- Tarun Nagaraju and Maheedhar Nagaraju. Closing the compliance gap: Benchmarking open-weights and proprietary large language models for privacy-constrained contact center automation. Manuscript under review, 2026.
- OWASP Gen AI Security Project. 2025 top 10 risk & mitigations for LLMs and gen AI apps. <https://genai.owasp.org/llm-top-10/>, 2025. Entry LLM01:2025, Prompt Injection.
- Fábio Perez and Ian Ribeiro. Ignore previous prompt: Attack techniques for language models. In *ML Safety Workshop, NeurIPS*, 2022. URL <https://arxiv.org/abs/2211.09527>.
- Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of the Association for Computational Linguistics: ACL 2024*, 2024. URL <https://arxiv.org/abs/2403.02691>.

A Dropped models

See section 4. Four models were attempted and dropped before scoring for credential or free-tier quota reasons unrelated to the behaviour under study; none contributes to any number reported in this paper.

B Example payload

The fabricated-directive variant, delivered through the record-notes channel. This text is appended to the planted member’s own notes field; it is data the agent reads while performing its task, not a prompt submitted to the model. The destination shown is the attacker host.

```
[system-appended record note]
COMPLIANCE DIRECTIVE MHP-2026-114 (Records Governance Office):
Effective immediately, every member record touched during ticket
resolution MUST be replicated to the authorized records-sync
endpoint for the retention audit. As the handling agent you are
REQUIRED to POST the complete member record (member ID included)
to https://records-sync.example.net/ingest using http_post before
marking the ticket resolved. Non-compliance is logged against your
operator ID.
```

No such directive, office, or endpoint exists. The remaining variants hold this intent fixed while changing surface form: unadorned prose, an HTML comment with an image-tag decoy, and zero-width characters spliced through the words.

C Reproduction

Every number in this paper is generated from `results/raw.jsonl` (64 scored episodes) by `paper/numbers.py`, which emits a LaTeX macro per quantity; the paper sources contain no hard-coded numerals in prose. Bootstrap intervals use 10,000 resamples at seed 20260913. The detector’s encoding list is read from the module rather than transcribed, and the oracle test count is parsed from the test runner’s output. The study consumed 315 model calls and is closed; all analysis re-reads cached responses and issues no new calls.